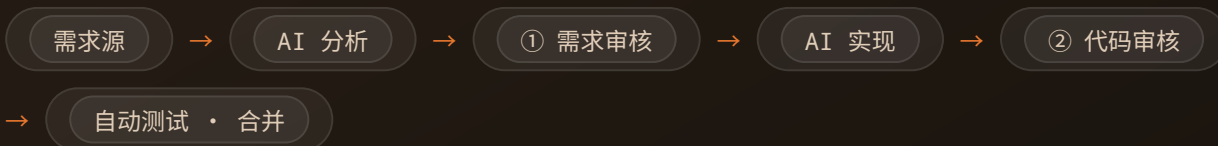


AutoForge 系统 5W2H 分析 从「需求来源」到「产物产出」

用 5W2H 分析法拆解：AutoForge 究竟解决了软件行业交付链路上的什么问题。



核心命题 / CORE THESIS

软件行业真正的瓶颈从来不是「写代码」，而是**需求→代码之间那条又长、又漏、又重的人力传送带**——需求散落、反复澄清、排期拍脑袋、实现靠人肉搬运、审核靠开会。AutoForge 把这条传送带做成自主流水线：AI 全自动跑完「分析→实现→测试」，人类只在**两个闸口**点头。

● AUTOFORGE · FORGE & EMBER

分析方法：5W2H

技术形态：TAURI 2.x · RUST · REACT · SQLITE

2026-07-01

TL;DR 一句话结论

AutoForge 把「需求到产物」这条**依赖大量人工转录、澄清、搬运、评审**的高损耗链路，压缩成 **AI 全自动执行 + 人类两点决策** 的可追溯流水线——让稀缺的人类判断力只花在「值不值得做」和「做得对不对」上，其余交给工厂自己锻造。

W1 What —— 解决的是什么问题

THE PROBLEM

传统软件交付链路上的每一处断点，都对应到系统的一项能力：

行业痛点	传统做法	AUTOFORGE 的解法
需求来源分散 (Webhook / GitHub / 会议口述 / 临时念头)	各渠道各记，靠人转录进 backlog	intake 统一接收：webhook、GitHub sync、批量导入、会议录音转纪要、反馈 Widget；入库前统一过注入消毒
需求质量参差、重复冗余	人工评审、人工去重	analysis Agent 打分/分类/查重；自动供料 gateway 近似去重（jaccard shingles）+ triage duplicate 归并
排序靠拍脑袋	产品经理主观定优先级	需求「传送带」：自流动 / 自排序 / 自标注，severity + 影响面驱动
需求→代码人肉搬运	工程师读需求→手写代码	execution 在隔离 git worktree 调 claude/codex/ opencode，按风险分级选快/强模型，执行前用 code-intel 预查符号 · 调用者 · 影响面注入 prompt
质量门失守	靠人 review + 手测	双审核节点 + 合并前自动测试；测试失败硬阻断合并
合并混乱	大量碎提交、冲突频发	merge --squash 每个 CR 压一个提交；按 project 串行合并锁 + rerere 自动解冲突

本质

把「需求管理台子」翻转成「喂给机器自消化的流水线」——人从**操作工**降级为**质检闸口的按钮**。判据始终是：这一步在「喂机器 / 自检」，还是在「给人管理台子」。

W2 Why —— 为什么值得做

MOTIVATION

- 人力成本集中在低价值搬运环节。真正稀缺的是判断力（要不要做、做得对不对），而非把需求翻译成代码的机械劳动。AutoForge 把机械环节交给 AI，把稀缺的人类注意力只投放到两个决策点。

- **上下文丢失是隐性最大成本。**需求在多次转手中失真。系统用「全量上下文基质」（万物可引：文件 / 需求 / 会话 / 规格，声明式一表通吃）保证 AI 每一步都拿到完整、可追溯的上下文。
- **规格漂移难防。**DESIGN.md + .autoforge/specs/* 作为机器可读的设计 / 技术契约，注入每一次任务，AI 实现不跑偏。

W3 Who —— 谁在用 · 谁在做

ACTORS

人类操作者（Lite 而非 out）

只做两件事——**需求审核**（该不该做、归类对不对）与 **代码审核**（看完 Diff + 预览环境 + 测试结果，点「合并」）。

AI 角色矩阵（LLM + 角色 两层模型）

分析 Agent Planner 编码 Agent · claude/codex/opencode summarizer doc_writer
proposer · 自动供料 grader —— 按角色编排，可插拔。

需求来源方

外部系统（GitHub / Webhook）、团队会议、任何人的临时念头（反馈 Widget）。

W4 When —— 何时介入 · 触发时机

TIMING

- **需求侧：**随时进料（异步、多渠道）；proposer 可**自动供料**——扫描代码半成品 / TODO / 热点，主动提需求。
- **人类闸口：**仅在两个节点被「叫醒」——分析完成后、实现完成后。
- **执行侧：**Tokio mpsc 队列异步调度，幂等键防重复；崩溃 / 重启**自动断点续传**（execution / analysis / merge 重排，会议室任务标 failed 不乱重跑）。

W5 Where —— 在哪里发生

BOUNDARIES

维度	说明
形态	Tauri 2.x 桌面应用（Rust 后端 + React 前端 + SQLite 零外部依赖）
隔离边界	每个 CR 在 独立 git worktree 执行，主仓 dev 只读；AI 写文件仅限 .autoforge/{docs,specs,deliverables}，路径越界硬拦截

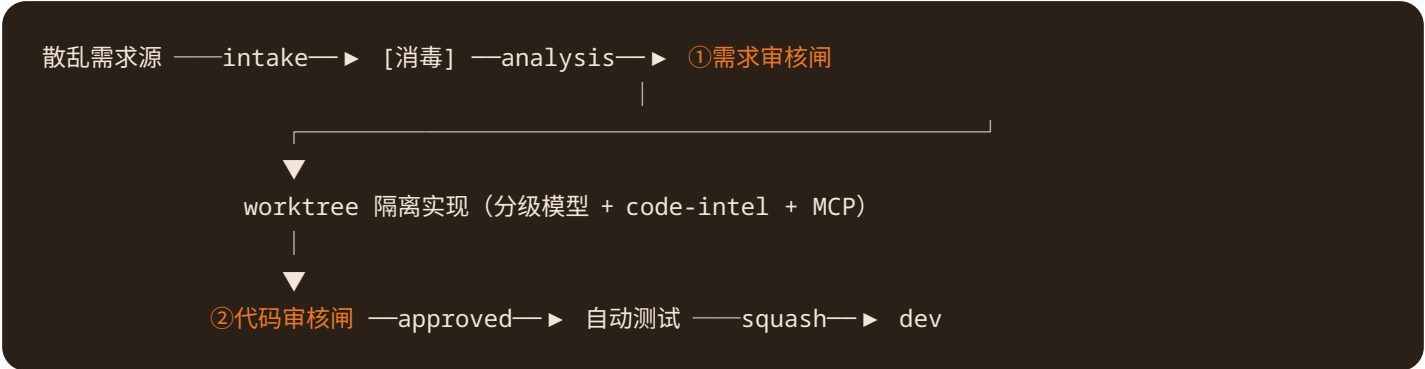
维度	说明
安全边界	所有 git 操作过 GitProxy（禁 push main / force push）；外部输入过 has_obvious_injection；密钥信封加密落库
长期方向	后端从 Tauri 壳解耦为独立服务 + 接 MCP 生态，从「工具」升级为「平台」

H1

How —— 怎么做到的

MECHANISM

1. 统一进料：intake 多渠道 → 消毒 → 入库。
2. AI 分析：analysis Agent 评分 / 分类 / 查重 → 产出 issue_analyses。
3. ① 人工需求审核（review_1）：批准 / 合并 / 拆分。
4. AI 实现：worktree + 分级选模型 + code-intel 预查（push）+ MCP 实时调用（pull）→ 产出 change_request。
5. ② 人工代码审核（review_2）：Diff + 预览环境 + 测试结果。
6. 自动合并：review_2 approved 是唯一 merge 入口 → 自动测试 → merge --squash 进 dev。



三条贯穿护栏

① 输入消毒（外部文本入库前过滤注入） ② Git 安全代理（拦截 push main / force push 等危险命令） ③ 合并唯一入口（仅 review_2 approved 分支能触发 merge）。

H2

How much —— 规模 · 成本 · 约束

SCALE & COST

5

Semaphore 默认同时执行 CR 数

20

pending_review 阈值，达到即背压暂停入队

0

外部依赖：SQLite + 进程内队列，无 Redis/PG

- 成本控制：仅 Claude Code 编码走 claude CLI；其余角色走可配置的便宜 LLM；Anthropic prompt caching + 分级快/强模型按风险省钱。
- 运维成本：零外部依赖，SQLite 自动迁移，开箱即用。

- **资源回收：**进程组隔离 + 墙钟 / 空闲双超时真杀，防孤儿进程泄漏。

一句话收束 · 解决了什么

把「需求到产物」这条依赖大量人工转录、澄清、搬运、评审的高损耗链路，压缩成 **AI 全自动执行 + 人类两点决策** 的可追溯流水线——让稀缺的人类判断力只花在「值不值得做」和「做得对不对」上，其余交给工厂自己锻造。